

Utilizando *um set*

O **set** (conjunto) guarda valores **sem repetição** e **sem ordem**. É a ferramenta certa para tirar duplicados, conferir se algo está numa coleção grande e comparar grupos: quem está nos dois, quem está só num, quem está em algum. Nesta aula: criar, alterar, as operações de conjunto e os dois erros que mais aparecem.

{ } e set()

Sem repetidos

Sem ordem

add · discard · remove

| & - ^

Subconjunto

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Sets; Built-in Types: Set Types), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Valores únicos, *sem ordem*

criar.py

```
cores = {"azul", "verde", "azul", "rosa"} # sem repetidos
print(cores)                             # a ordem pode variar
print(len(cores))
print("verde" in cores)

vazio = set() # {} sozinho cria um Dicionário vazio
print(type(vazio), type({}))
```

SAÍDA

```
{'verde', 'azul', 'rosa'}
3
True
<class 'set'> <class 'dict'>
```

- ✓ Chaves `{ }` com valores soltos (sem `chave: valor`)
- ✓ O "azul" repetido entrou **uma vez só**
- ✓ A ordem da saída **não** é a ordem em que você escreveu, e pode mudar de uma execução para outra
- ✓ `in` num set é **muito rápido**, mesmo com milhões de itens

`{ }` vazio cria um **dicionário** (aula 12). Set vazio é `set()` .

Tirando *os repetidos*

sem_repetidos.py

```
visitas = ["ana", "bia", "ana", "caio", "bia", "ana"]
unicos = set(visitas)
print(len(visitas), "visitas,", len(unicos), "pessoas")
print(sorted(unicos))    # sorted devolve uma lista em ordem
```

SAÍDA

```
6 visitas, 3 pessoas
['ana', 'bia', 'caio']
```

- ✓ `set(lista)` elimina os repetidos de uma vez
- ✓ `len()` do set = quantos valores **diferentes**
- ✓ `sorted(conjunto)` devolve uma **lista** em ordem, para mostrar

Se a ordem original importa, `list(dict.fromkeys(lista))` tira repetidos mantendo a ordem de chegada (dicionários guardam a ordem).

Adicionando *e removendo*

alterar.py

```
tags = {"python"}
tags.add("cursor")           # adiciona um
tags.update(["ia", "mcp"])   # adiciona vários
tags.add("python")          # já existe: nada muda
print(sorted(tags))
tags.discard("java")         # remove se existir (sem erro)
tags.remove("mcp")           # remove (dá erro se não existir)
print(sorted(tags))
```

SAÍDA

```
['cursor', 'ia', 'mcp', 'python']
['cursor', 'ia', 'python']
```

MÉTODO	FAZ
<code>.add(x)</code>	Adiciona um valor (se já existe, nada muda)
<code>.update(itens)</code>	Adiciona vários
<code>.discard(x)</code>	Remove, sem erro se não existir
<code>.remove(x)</code>	Remove; KeyError se não existir
<code>.pop()</code>	Remove e devolve um valor qualquer
<code>.clear()</code>	Esvazia

Comparando *grupos*

operacoes.py

```
python = {"ana", "bia", "caio", "davi"}
dados = {"bia", "davi", "eva"}
print(sorted(python | dados))    # união: estão em algum dos dois
print(sorted(python & dados))    # interseção: estão nos dois
print(sorted(python - dados))    # diferença: só em python
print(sorted(python ^ dados))    # só em um dos dois
print({"bia"} <= dados)         # é subconjunto?
```

SAÍDA

```
['ana', 'bia', 'caio', 'davi', 'eva']
['bia', 'davi']
['ana', 'caio']
['ana', 'caio', 'eva']
True
```

OPERADOR	MÉTODO	RESULTADO
a b	a.union(b)	União: em algum dos dois
a & b	a.intersection(b)	Interseção: nos dois
a - b	a.difference(b)	Só em a
a ^ b	a.symmetric_difference(b)	Em um só
a <= b	a.issubset(b)	Tudo de a está em b?

Exemplos reais: alunos de duas turmas, produtos de duas lojas, permissões de dois perfis, tags de dois posts.

Os dois *erros mais comuns*

`erro_indice.py` · SET NÃO TEM POSIÇÃO

```
cores = {"azul", "verde"}  
print(cores[0])
```

```
Traceback (most recent call last):  
  File "erro_indice.py", line 2, in <module>  
    print(cores[0])  
          ~~~~~^^^  
TypeError: 'set' object is not subscriptable
```

Sem ordem, não há "primeiro". Para acessar por posição, converta: `sorted(cores)[0]` .

`erro_lista.py` · LISTA DENTRO DO SET

```
grupos = {"ana", "bia"}
```

```
Traceback (most recent call last):  
  File "erro_lista.py", line 1, in <module>  
    grupos = {"ana", "bia"}  
              ^^^^^^^^^^^^^  
TypeError: cannot use 'list' as a set element (unhashable type: 'list')
```

Itens de set precisam ser imutáveis ("hashable"). Lista não pode; **tupla** pode (aula 10): `{"ana", "bia"}` .

Set, lista *ou tupla?*

	LIST	TUPLE	SET
Sintaxe	[1, 2]	(1, 2)	{1, 2}
Ordem	Sim	Sim	Não
Repetidos	Sim	Sim	Não
Alterar	Sim	Não	Sim (frozenset: não)
Índice [0]	Sim	Sim	Não
Busca com in	Lenta em listas grandes	Lenta em tuplas grandes	Muito rápida

Pergunte: "a ordem importa? pode repetir?" Se as duas respostas são **não**, é set.

Quem fez *as duas provas*?

Enunciado: com as listas de presença das duas provas (com nomes repetidos), mostre quem fez as duas, quem fez só a primeira e o total de alunos diferentes.

Uma resposta possível

```
# Exercício: quem fez as duas provas?
prova1 = ["ana", "bia", "caio", "ana", "davi"]
prova2 = ["bia", "davi", "eva", "bia"]
p1, p2 = set(prova1), set(prova2)
print("Fizeram as duas:", sorted(p1 & p2))
print("Só a primeira:", sorted(p1 - p2))
print("Total de alunos:", len(p1 | p2))
```

SAÍDA

```
Fizeram as duas: ['bia', 'davi']
Só a primeira: ['ana', 'caio']
Total de alunos: 5
```

- ✓ Converter para set já tira os repetidos
- ✓ `&`, `-` e `|` respondem as três perguntas
- ✓ Desafio: quem faltou a alguma das provas?

Teste o que *ficou*

1. Quanto vale `len({1, 2, 2, 3, 3, 3})`?

A) 6

B) 3

C) 1

D) Erro

B. Sem repetidos.

2. Como criar um set vazio?

A) `{}`

B) `set()`

C) `[]`

D) `()`

B. `{}` é dicionário.

3. `{1, 2, 3}` & `{2, 3, 4}` dá:

A) `{1, 2, 3, 4}`

B) `{2, 3}`

C) `{1, 4}`

D) `{1}`

B. Interseção.

4. Qual remove sem erro se o item não existir?

A) `.remove()`

B) `.discard()`

C) `.pop()`

D) `del`

B. `.remove()` dá `KeyError`.

Cada valor *uma vez só*

O set guarda valores **únicos** e **sem ordem**: tira repetidos, busca com `in` muito rápido e compara grupos com `|`, `&`, `-` e `^`. Set vazio é `set()`; não há índice; e os itens precisam ser imutáveis.

`set(lista)`

`add · discard`

`| & - ^`

`set()` vazio

Sem índice

Próxima aula → 12 · Utilizando um Dicionário

Fontes: docs.python.org/3/tutorial/datastructures.html#sets; docs.python.org/3/library/stdtypes.html#set-types-set-frozenset, consultados em outubro de 2026.
Exemplos em `exemplos/`, executados com Python 3.14.4 (saídas com PYTHONHASHSEED=0, para a ordem dos sets ficar estável).